ASSIGNMENT 3 - TASK 4

VIRTUAL MEMORY AND DEMAND PAGING

1. Introduction and Problem Context

The assignment describes a radiologist running an image-processing program that uses 24 GB of memory on a laptop with 16 GB of RAM. Demand paging allows a process to have a virtual address space larger than physical memory. Only pages currently needed are brought into RAM; other pages remain backed by storage until referenced. The key issue is whether the active working set fits in available memory or causes repeated faults and thrashing.

This report follows Q4 - Virtual Memory and Demand Paging (8 Marks, K5 - Evaluate), pages 26-27 of the supplied question paper. It covers page-fault handling, effective access time, copy-on-write, working sets, frame allocation, thrashing, sequential/random memory experiments, a fork() memory experiment, a frame-count plot and the 32 GB RAM alternative.

2. Page-Fault Handling

For a valid page that is not resident, the operating system typically performs these steps:

Step	Operating-system action
1. Trap	The MMU detects the not-present page and traps into the kernel with the faulting address and access type.
2. Validate	The kernel checks the process virtual-memory area and permissions. If invalid or prohibited, it reports an error (commonly SIGSEGV on Linux).
3. Locate backing data	The kernel identifies the backing source, such as an executable/file mapping or swap slot. A demand-zero page can be initialised as zero.
4. Obtain a frame	The kernel obtains a free physical frame or selects a victim page. A dirty victim may need to be written to backing storage.
5. Read or initialise	The needed page is read from storage or zero-filled into the selected frame.
6. Update mapping	Page-table metadata is updated and the TLB is updated or invalidated as required.
7. Restart	The faulting instruction is restarted and the process resumes.

Not every fault needs disk I/O. A minor fault can be handled without reading a page from storage; a major fault requires I/O such as loading a file or swap page. An invalid address is rejected rather than repaired by loading a page.

3. Effective Access Time (EAT)

Given memory access time M = 100 ns and page-fault service time F = 8 ms = 8,000,000 ns. After servicing a fault, the instruction is restarted and needs a memory access. Therefore:

EAT = (1 - p) * M + p * (F + M)
EAT = (1 - p) * 100 + p * (8,000,000 + 100) = 100 + 8,000,000p ns.

Page-fault rate	Calculation	Effective access time
1 in 1,000 = 0.001	100 + 8,000,000(0.001)	8,100 ns = 8.1 microseconds
1 in 100,000 = 0.00001	100 + 8,000,000(0.00001)	180 ns
1 in 1,000,000 = 0.000001	100 + 8,000,000(0.000001)	108 ns

Even a small page-fault probability can dominate access time because an 8 ms service is many orders of magnitude slower than a 100 ns memory access.

4. Maximum Page-Fault Rate for a 10% Slow-Down

A 10% slow-down permits EAT to rise from 100 ns to at most 110 ns:

100 + 8,000,000p <= 110
8,000,000p <= 10
p <= 0.00000125 = 1.25 x 10^-6

The maximum acceptable rate is approximately one fault per 800,000 memory references. Above this rate, mean memory-access time exceeds the 10% slow-down target under the stated assumptions.

5. Demand Paging: 24 GB Program on 16 GB RAM

The 24 GB figure is the program's virtual-memory requirement; it does not mean every byte must be resident in physical RAM at once. The process can reserve a 24 GB virtual address space while only the pages it actively touches are loaded into the 16 GB RAM. Non-resident pages can remain backed by executable files, mapped files or swap and are loaded on demand when referenced.

This works efficiently when the active working set is smaller than the physical memory available after the OS and other processes are accounted for. If the active working set repeatedly exceeds available frames, pages are evicted and reloaded frequently. The process then thrashes and runs very slowly.

6. Copy-on-Write and Fast fork()

With copy-on-write (COW), fork() initially creates child mappings to the same physical pages as the parent. The OS marks mappings so that writes trigger a protection fault and a private page copy is created only when modified. Untouched pages remain shared.

This makes fork() faster and more memory-efficient than immediately copying the entire address space. Initial work mainly creates process metadata and page-table structures. If the child later modifies many pages, those pages must be copied, so the saving depends on how much memory is written.

cow_fork_demo.c maps and touches 64 MiB, reads RSS/PSS and shared/private dirty values from /proc/self/smaps_rollup, forks, measures the child before writing, writes one byte per page and measures again. After fork, shared-dirty memory rises and PSS is divided between processes; after child writes, Private_Dirty rises because COW copies have been created. RSS alone is not sufficient to quantify sharing because shared pages can be counted in both processes' RSS.

7. Thrashing and the Working-Set Model

Thrashing occurs when processes spend most of their time servicing page faults and moving pages between RAM and backing storage instead of executing useful work. It commonly occurs when total active working-set demand exceeds the frames available to active processes.

The working set of process i, WSS_i, is the set of distinct pages referenced during a recent window Delta (a fixed number of references or a time interval). Total demand is D = sum(WSS_i). If D exceeds the available frames, some working sets cannot remain resident and page faults rise.

The OS can observe high page-fault frequency and paging I/O together with low useful CPU utilisation. It can give a process more frames, reduce multiprogramming by suspending or swapping out a process, or use page-fault-frequency thresholds to adjust frame allocation. Working-set-based allocation tries to keep each active process's current working set resident where possible.

8. Frame Allocation: Equal and Proportional

Policy	Method	Evaluation
Equal allocation	With m frames and n processes, allocate approximately m/n frames per process, subject to minimums and rounding.	Simple, but ignores process size and working-set demand.
Proportional allocation	Process i of size s_i receives approximately (s_i / sum(s_j)) * m frames.	Reflects process size, but size may not match current locality.
Working-set/dynamic allocation	Estimate each process's current WSS and allocate enough frames to cover it when possible.	Tracks active locality better, but adds tracking overhead. If demand exceeds capacity, reduce multiprogramming.

A fixed equal or size-proportional share can be poor if actual locality differs substantially from process size. Allocation should adapt as working sets change.

9. Sequential and Random Memory Access Experiment

memory_access_bench.c allocates a fresh anonymous mapping and touches one byte in every native page. It runs sequential and deterministic shuffled orders, then revisits the working set for three passes. The program records elapsed time and minor/major page-fault deltas with getrusage(). Both tests use the same working-set size and page count.

A first touch to a fresh anonymous page usually creates a minor fault, so sequential and random tests can show similar fault counts when each page is visited. Runtime can differ because of locality, cache and TLB behaviour. If the working set exceeds the RAM allowance in a VM and evicted pages are revisited on later passes, swap activity and major faults may appear. Results depend on the kernel, swap and competing workload.

Working set	Order	Runtime (ms)	Minor faults	Major faults
32 MiB	Sequential	Local output	Local output	Local output
32 MiB	Random	Local output	Local output	Local output
64 MiB	Sequential	Local output	Local output	Local output
64 MiB	Random	Local output	Local output	Local output
128 MiB	Sequential	Local output	Local output	Local output

Working set	Order	Runtime (ms)	Minor faults	Major faults
128 MiB	Random	Local output	Local output	Local output

The default memory sweep is conservative and skips sizes above half of MemAvailable. Fill the table using the output generated on the user's own machine or VM.

10. Measuring Beyond Available RAM Safely

The assignment permits a virtual machine or a memory limit. Use a dedicated VM with fixed RAM and suitable swap, then increase the test working set gradually so it approaches and exceeds available VM memory. Record RAM, MemAvailable and swap before each run. The script defaults to 32, 64 and 128 MiB and includes a safety guard. ALLOW_MEMORY_PRESSURE=1 bypasses the guard and should only be used inside a dedicated VM after checking total RAM-plus-swap capacity.

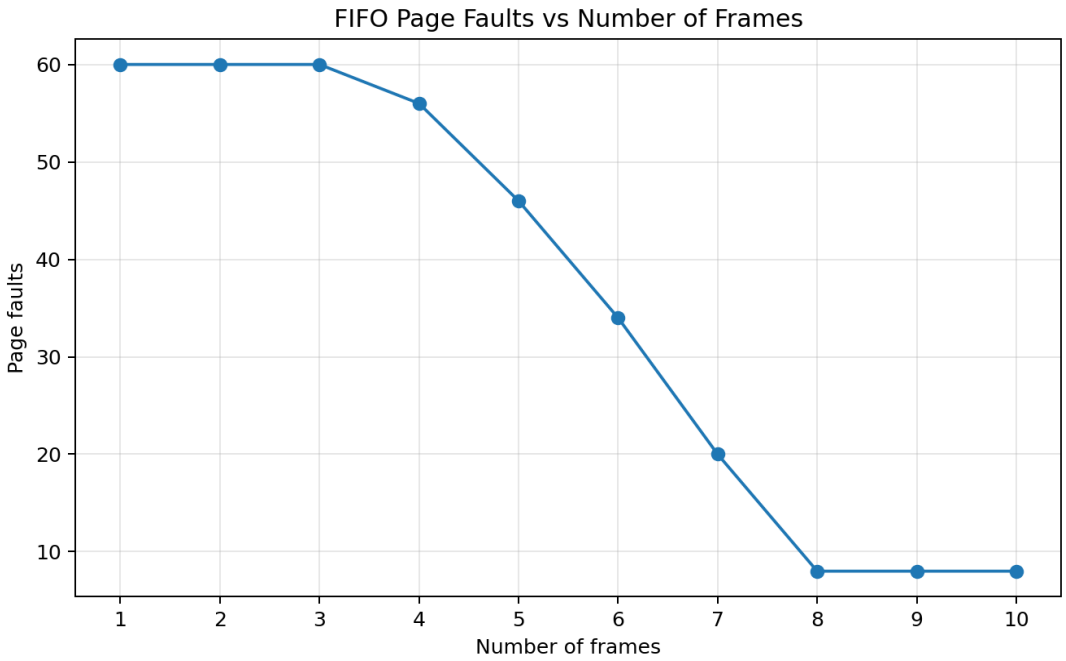
Example only for a VM configured with sufficient RAM and swap:

```
MEM_SIZES_MB="256 512 1024 2048 3072" ALLOW_MEMORY_PRESSURE=1 ./run_task4.sh
```

Do not run aggressive memory-pressure tests on the everyday Windows/WSL host. If the process exceeds the VM's combined RAM and swap, it may be killed by the OOM mechanism. If no VM is available, report the safe local sweep and state that the beyond-available-RAM condition was not tested.

11. Page Faults Against Number of Frames

frame_faults.c evaluates FIFO for a fixed 60-reference string with repeated locality around pages 1-8. It counts faults for 1-10 frames and writes frame_faults.csv. plot_faults.py reads this CSV and produces frame_faults.png.



The plot demonstrates how frame count affects faults for this specified FIFO string. It is a controlled reference-string experiment, not a trace from the radiology application.

12. Alternative: 32 GB RAM Instead of 16 GB

With 32 GB RAM, more of the 24 GB virtual working set may remain resident, reducing faults if the workload touches most of those pages and enough memory remains for the OS and other applications. If the active working set fits in available RAM, the fault rate after warm-up may approach zero and EAT approaches 100 ns. More RAM does not eliminate first-touch demand loading, file I/O, invalid accesses or a working set larger than 32 GB.

Using representative measured page-fault rates p_16 and p_32:

EAT_16 = 100 + 8,000,000p_16 ns
EAT_32 = 100 + 8,000,000p_32 ns

Metric	16 GB system	32 GB system
Page-fault rate	Measure p_16	Measure p_32
Effective access time	100 + 8,000,000p_16 ns	100 + 8,000,000p_32 ns
Memory purchase cost	Baseline	Additional cost of extra memory
Performance	Measured runtime	Compare runtime and time saved

The assignment does not supply the imaging application's trace, measured fault rate or market price of memory. Those values must be measured or quoted for the actual laptop. Improving locality is usually the first software step because it reduces the working set and faults without hardware cost. Additional RAM is justified when the active working set remains larger than available memory after reasonable locality improvements and the measured time saved is worth the price.

13. Conclusion

Demand paging permits a 24 GB virtual-memory program to run on a 16 GB laptop when only a subset of pages is actively resident. Under the supplied 100 ns memory and 8 ms fault-service assumptions, the fault rate must stay at approximately one per 800,000 references or lower to keep slow-down within 10%. COW makes fork() efficient by sharing pages until they are written. Working-set and page-fault-frequency methods help detect thrashing and guide frame allocation. The experiments provide sequential/random access measurements, COW statistics and a frame-fault plot. The final RAM-versus-locality choice must use representative measurements from the actual imaging workload.

14. Execution Procedure and Evidence

1. Place memory_access_bench.c, cow_fork_demo.c, frame_faults.c, plot_faults.py and run_task4.sh in one Task04 folder.
2. Open Ubuntu/WSL in that folder.
3. Run chmod +x run_task4.sh.
4. Run ./run_task4.sh for safe default measurements.
5. For beyond-available-RAM testing, use a dedicated memory-limited VM with swap and follow README.md before enabling ALLOW_MEMORY_PRESSURE=1.
6. Review memory_access_results.txt and cow_fork_results.txt.
7. Review frame_faults.csv and frame_faults.png.
8. Capture screenshots of actual terminal outputs and preserve generated files.

Evidence checklist: page-fault handling explanation; three EAT calculations; 10% maximum fault rate; COW memory measurements; sequential/random page-fault and runtime measurements; controlled beyond-free-RAM experiment; FIFO faults vs frames plot; thrashing/working-set and frame-allocation analysis; and the 32 GB alternative.

Source alignment: Q4 - Virtual Memory and Demand Paging, 8 Marks, K5 - Evaluate, pages 26-27 of the supplied Assignment 3 question paper.